

Class 6: R Functions

Brooke Clements (PID: A17532793)

Table of contents

Background	1
Our first function	1
A Protien generating function	5

Background

All functions in R have at least 3 things:

- A **Name** that we use to call the function.
- Has one or more input **Arguments**
- The **Body** the lines of R code that do the work

Our first function

Let's write a silly wee function called `add()` to add some numbers (the input arguments)

```
add <- function(x,y) {x+y}
```

```
add(100,1)
```

```
[1] 101
```

```
add(x=c(100,1,100),y=1)
```

```
[1] 101    2 101
```

Q. What if I give a multiple element vector to `x` and `y`?

```
add(x=c(100,1),y=c(100,1))
```

```
[1] 200    2
```

Q. What if I give three inputs to the function?

```
#add(x=c(100,1),y=1,z=1)
```

Q. What if I give only one input to the add function?

```
addnew <- function(x,y=1) {x+y}
```

```
addnew(x=100)
```

```
[1] 101
```

```
addnew(c(100,1),100)
```

```
[1] 200 101
```

If we write our function with input arguments having no default value then the user will be required to set them when they use the function. We can give our input arguments “default” values by setting them equal to some sensible value -e.g. $x=1$, $y=1$

Let’s try something more interesting: Make a sequence generating tool..

The `sample()` function can be a useful starting point here:

```
sample(1:10, size=4)
```

```
[1]  8 10  3  7
```

Q. Generate 9 random numbers taken from the input vector $x=1:10$?

```
sample(1:10, size=9)
```

```
[1]  7  3 10  8  1  2  6  5  4
```

Q. Generate 12 random numbers taken from the input vector $x=1:10$?

```
sample(1:10, size=12, replace=TRUE)
```

```
[1] 2 8 10 4 7 6 4 4 3 5 7 7
```

Q. Write code for the `sample()` function that generates nucleotide sequences of length 6?

```
sample(x=c("A","G","C","T"), size=6, replace=TRUE)
```

```
[1] "G" "T" "G" "A" "C" "C"
```

Q. Write a first function `generate_dna()` that returns a *user specified length* DNA sequence:

```
generate_dna <- function(len=6){sample(x=c("A","G","C","T"), size=len, replace=TRUE)}
```

```
generate_dna(len=100)
```

```
[1] "C" "A" "C" "T" "G" "C" "A" "C" "A" "A" "C" "A" "C" "G" "C" "C" "A" "T"
[19] "C" "A" "T" "A" "C" "G" "C" "A" "G" "C" "T" "T" "C" "A" "T" "T" "A" "C"
[37] "A" "T" "T" "A" "T" "T" "A" "G" "C" "T" "T" "G" "T" "C" "T" "G" "G" "C"
[55] "G" "C" "T" "T" "C" "C" "A" "A" "T" "T" "C" "C" "C" "A" "C" "T" "A" "C"
[73] "A" "A" "A" "C" "T" "C" "C" "G" "C" "T" "A" "A" "G" "C" "G" "G" "A" "C"
[91] "A" "A" "G" "A" "G" "C" "G" "C" "C" "T"
```

Key-Points Every function in R looks fundamentally the same in terms of its structure. Basically 3 things: name, input, and body.

```
name <- function(input) {body}
```

Functions can have multiple inputs. These can be **required** arguments or **optional** arguments. With optional arguments having a set default value.

Q. Modify and improve our `generate_dna()` function to return its generated sequence in a more standard format like “AGTAGTA” rather than vector “A”, “C”, “G”, “A”

```

generate_dna <- function(len=6, fasta=TRUE){
  ans <- sample(x=c("A","G","C","T"),
               size=len, replace=TRUE)
  if(fasta){
    cat("Single-element vector output")
    ans <- paste(ans, collapse="")
  }else{
    cat("Multi-element vector output")
  }
  return(ans)
}

generate_dna (fasta=FALSE)

```

Multi-element vector output

```
[1] "A" "A" "C" "A" "T" "T"
```

```
generate_dna (fasta=TRUE)
```

Single-element vector output

```
[1] "CGGGGA"
```

The `paste` function - it's job is to join up or stick together (a.k.a paste) input strings together

```
paste("alice","loves R",sep="***")
```

```
[1] "alice***loves R"
```

Flow control means where the R brain goes in your code

```

good_mood <- TRUE

if(good_mood){
  cat("Great!")
} else {
  cat("Bummer!")
}

```

Great!

A Protein generating function

Q. Write a function, called `generate_protein()`, that generates a user specified length protein sequence.

Q. Use that function to generate random protein sequences between length 6 and 12

Q. Are any of your sequences unique i.e not found anywhere in nature?

There are 20 natural amino-acids

```
aa <- c("A","R","N","D","C","Q","E","G","H","I","L","K","M","F","P","S","T","W","Y","V")
```

```
generate_protein <- function(len){  
  #The amino-acids to sample from  
  aa <- c("A","R","N","D","C","Q","E","G","H","I","L","K","M","F","P","S","T","W","Y","V")  
  #Draw n=len amino acids to make our sequence  
  ans <- sample(aa,size=len,replace=T)  
  ans <- paste(ans, collapse="")  
  return(ans)  
}
```

```
generate_protein(20)
```

```
[1] "MLSNCRWSTHRGINWKLDES"
```

Q. Use that function to generate random protein sequences between length 6 and 12

```
generate_protein(6)
```

```
[1] "QYCCPK"
```

```
generate_protein(7)
```

```
[1] "QKGCVCT"
```

```
generate_protein(8)
```

```
[1] "VFEIADEH"
```

```
generate_protien(9)
```

```
[1] "RIACMCNCL"
```

```
generate_protien(10)
```

```
[1] "NRMEEVGQCH"
```

```
generate_protien(11)
```

```
[1] "VANHWFPHFNF"
```

```
generate_protien(12)
```

```
[1] "DTTTSNSASNFY"
```

```
for(i in 6:12){  
  #FASTA ID line">id"  
  cat(">",i, sep="", "\n")  
  #Protein sequence line  
  cat(generate_protien(i),"\n")  
}
```

```
>6  
CASHDD  
>7  
CQRAAPK  
>8  
VYFNCCKY  
>9  
SHGWIYGGP  
>10  
EHLHQPDMN  
>11  
LDGRCTRFCNE  
>12  
KPYPAPPPKDW
```

Q. Are any of your sequences unique i.e not found anywhere in nature?

Sequences 6 -8 were found in nature but 9-12 were not found in nature.